Alumni Web App

A SOFTWARE PROJECT REPORT
submitted for the partial fulfilment of the award of the degree of
MASTER OF COMPUTER APPLICATIONS

BY

Mohammed Salman M

BP231013

Under the Guidance of

Asst.Prof. DR. S. Anthony Philomen Raj M.C.A, Ph. D.

PG DEPARTMENT OF COMPUTER APPLICATION

SACRED HEART COLLEGE (AUTONOMOUS)

TIRUPATTUR DT-635 601

April 2025

BONAFIDE CERTIFICATE

This is to certify that the project entitled **"ALUMNI WEB APP"** being submitted to Sacred Heart College (Autonomous), Tirupattur by **MOHAMMED SALMAN M (BP231013)** for the partial fulfilment for the award of a degree in Master of Computer Applications is a project work carried out under my guidance and supervision.

Signature of the HOD

Signature of the Guide

Submitted for the viva-voce examination held on: _____

Examiner

1:

2:

Department of Computer Applications (PG)

SACRED HEART COLLEGE (Autonomous)

Tirupattur- 635 601, Tirupattur District, Tamil Nadu

CERTIFICATE OF ORIGINALITY

(Plagiarism Tested)

It is certified that the MCA project **ALUMNI WEB APP**

By **M. MOHAMMED SALMAN (BP231013)** has been examined by us and we undertake the following:

I. No sentence, diagram, table, paragraph, or section has been copied verbatim from previous work unless it is placed under quotation marks and duly referenced

II. The project work has not presented any others as their own, and has taken steps to ensure the ethical and academic integrity of their work

III. The project has been checked using TURNITIN (copy of originality report attached as an Annexure at the end of the project) and found within limits (less than 30%) as per Sacred Heart College Plagiarism policy

Place: Tirupattur

Date: _____

Asst. Prof. DR. S. Anthony Philomen Raj **Dr. A. George Louis Raja**

Project Guide: Head of the Department

BP231013_Alumni_Web_App.
do cx

by BP231013 MOHAMMED SALMAN M

Submission date: 13-Mar-2025 12:48PM (UTC+0530)
Submission ID: 2613311494
File name: BP231013_Alumni_Web_App.docx (4.61M)
Word count: 7876
Character count: 48275

BP231013_Alumni_Web_App.docx

ORIGINALITY REPORT

8%	5%	5%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	cors.archive.org Internet Source	1%
2	Submitted to University of Stirling Student Paper	1%
3	lib.buet.ac.bd:8080 Internet Source	1%
4	gitlab.isb-sib.ch Internet Source	1%
5	Submitted to University of Huddersfield Student Paper	<1%
6	Submitted to Southern New Hampshire University - Continuing Education Student Paper	<1%
7	Clarence Ho. "Pro Spring 3", Springer Science and Business Media LLC, 2012 Publication	<1%
8	Submitted to University of Sydney Student Paper	<1%

javaonfly.blogspot.com

9	Internet Source	<1%
10	Submitted to International School of Management and Technology Student Paper	<1%
11	text-id.123dok.com Internet Source	<1%
12	Submitted to Sunway Education Group Student Paper	<1%
13	Submitted to University of Bolton Student Paper	<1%
14	www.ijert.org Internet Source	<1%
15	www.coursehero.com Internet Source	<1%
16	www.daniweb.com Internet Source	<1%
17	Submitted to Arab Open University Student Paper	<1%
18	Submitted to University of Sunderland Student Paper	<1%
19	Banu Parasuraman. "Mastering Spring AI", Springer Science and Business Media LLC, 2024 Publication	<1%

20	Submitted to University of Essex Student Paper	<1%
21	Submitted to University of Teesside Student Paper	<1%
22	Submitted to University of the West Indies – ROYTEC Student Paper	<1%
23	Submitted to aou Student Paper	<1%
24	onjoraclejava.blogspot.com Internet Source	<1%
25	Submitted to Universiti Teknikal Malaysia Melaka Student Paper	<1%
26	Submitted to University of Bedfordshire Student Paper	<1%
27	mobidev.biz Internet Source	<1%
28	Marten Deinum, Daniel Rubio, Josh Long. "Chapter 8 Spring Batch", Springer Science and Business Media LLC, 2023 Publication	<1%
29	github.com Internet Source	<1%

www.c-sharpcorner.com

30	Internet Source	<1%
31	www.fastercapital.com Internet Source	<1%
32	artisansweb.net Internet Source	<1%
33	shctpt.edu Internet Source	<1%
34	www.jeejava.com Internet Source	<1%
35	www.justice.gov Internet Source	<1%
36	Felipe Gutierrez. "Pro Spring Boot 3", Springer Science and Business Media LLC, 2024 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		

ACKNOWLEGEMENT

I thank God Almighty for the blessings and grace by which I was able to complete this project work successfully. I sincerely thank my parents, unless and my relatives for their continuous support, love and blessings for me to attain many achievements.

I take this opportunity to express my gratitude to those who helped me and led me to complete this project work. I offer my humble gratitude to **Rev. Dr. D. Maria Antony Raj Principal**, Sacred Heart College (Autonomous), Tirupattur for permitting me to do my project work. I would like to thank the Director, HR and the entire team at Queenbug Technologies for the guidance they provided to me to complete this project successfully.

I express my deep gratitude to **Dr. S. Sagayaraj** Director, Department of Computer Applications (PG), Sacred Heart College (Autonomous), Tirupattur.

I would like to thank **Dr. A. George Louis Raja** Head, Department of Computer Applications (PG), Sacred Heart College (Autonomous) Tirupattur for providing the necessary arrangement for me to do my project work and training at Queenbug Technologies

My sincere thanks to **DR. S. Anthony Philomen Raj** internship Guide, who has guided me throughout my project work, I am grateful to him for his encouragement, guidance and assistance towards my project work.

.

MOHAMMED SALMAN M

(BP231013)

Contents

CHAPTERS	INDEX	PAGE-NO
CHAPTER - I	Abstract Introduction 1.1 Purpose 1.2 System Overview and Development Goal 1.3 Problem Statement 1.4 System Model 1.5 Project Time Line Diagram(Gant Chart)	**01-11**
CHAPTER - II	2.1 Module Identification and Description 2.2 User Role and Characteristic 2.3 Functional Requirement 2.4 Performance Requirement 2.5 Hardware and Software details (Implementation and Deployment Side) 2.6 Conceptual Diagram (DB entity relationship diagram)	**12-23**
CHAPTER - III	3.1 System Architecture Diagram 3.2 UML Design Models Use Case Diagram Class Diagram	**24-44**

	Activity Diagram Sequence Diagram 3.3 Database Design Models Table Structure Table Relationship Diagram Query Design 3.4 UI Design	
CHAPTER - IV	Implementation of Java Code	**45-49**
CHAPTER - V	5.1 Test Plan Scope of Testing Test Cases Types of Testing Testing Tools Testing Environment	**50-53**
CHAPTER - VI	Implementation and Deployment	**54**
CHAPTER - VII	7.1 I/O Report 7.2 Conclusion	**55-57**

CHAPTER – I

Abstract:-

An application that inherits all the features of the user college is the **Alumni App**, an up-to-date platform that links **alumni** with their colleges and provides them with needed tools for engagement and collaboration. This app is based on **Spring Boot** as its **backend** and built with **Bootstrap and Tailwind CSS**. Therefore, users should experience glitch-free experiences for **alumni** and **administration**. It is a **stand-alone network** for **Alumni** where they can interact with other **graduates**, **join in discussions**, surf through **job openings**, and keep informed about **events** happening in the institution. **Login** and **message** features, **event management**, and a mentorship-reserved space would allow alumni to contribute and, at the same time, benefit from a very active professional community.

Alumni Membership fees will be paid by **PayU integrated payments** with the members to ease interaction with contributing members.

The application provides **admin** with complete **dashboard** control of the **alumni engagement** into efficient methods. Admins log in securely to get general or graduated information about the alumni, to manage user roles, and effectively supervise some administrators. They have total power over the **news updates** and the **stories** told by alumni, thus assuring that such information shared is accurate, pertinent, and updated. Thus, by providing this connection for alumni with their institutions, **Alumni App** acts as a single-interface digital platform that boosts engagement, reinforces professional relationships, and advances careers. Strengthening relationships through meaningful interactions among mentors and peers, resource sharing, and mentorship opportunities internal to the app above creates a dynamic and virally well-knit alumni network within which everyone thrives, as does the institution.

Introduction :-

The Alumni App is a one-stop shop to create a totally automated seamless phenomenon, where alumni can engage and communicate with the institution and facilitate a stronger bond with the alma mater for life.

The app is built on the latest web technologies together with insight from data to create effective channels for networking, mentorship, and institutional support. .

- The Alumni Dashboard help administrators understand various data of alumni and become able to analyze trends like engagement, cohort-wise demographics, event part participation, and how to use decision-making ability based on data with alumni improvement in planning outreach programs.
- Network/mentorship creates a network of alumni who can mentor current students or fellow alumni to create a blanket community for deployments of professional development opportunities.
- Ensures smooth subscription, event registrations, and donations processing through PayU; guarantees easy and safe financial transactions with its membership and payment integration. Last but not least, the venture empowers institutions to make informed decisions, strengthening alumni relationships, and developing professional initiatives through the historical data of alumni. Finally, an authentic structure will be created to facilitate alumni engagement that will raise the institution's profile while simultaneously adding valuable opportunities for networking.

1.1 Purpose :-

This app provides a linked interface to ensure engagement of **alumni** into their respective institutions, as well as networking and professional development opportunities. It allows an alumnus to remain connected with former colleagues, participate in institutional events, and boost the growth of his/her college community. Communication, therefore, gets enhanced, allowing very meaningful interaction between alumni, students, and administrators.

On their side, college administrators are also furnished with efficient tools to manage alumni data, track engagement, and organize reunions or networking events. With integrated payment solutions like PayU, or Razorpay, alumni can pay easily for membership registration, event registration, and donations, thereby ensuring a good transaction experience.

The app is also a hub for career opportunities, mentorship programs, and industry collaborations, thereby facilitating alumni in moments of help for each other in the professional journey.Through technology to enhance alumni relations, the Alumni Web App cements institution-life bonds, improves engagement, and builds a thriving alumni network for the benefit of both an individual and the organizations.

1.2 Objectives :-

- ✓ **Make Better alumni networking** by creating a meaningful connection among alumni and current students, and facilitate mentoring, career guidance, and professional collaboration.

- ✓ **Alumni Management** in One Centralized Place-Provide college administrators with tools for organizing, tracking, and managing alumni data such as batch-wise categorization and event participation with complete efficiency and effectiveness.

- ✓ **Event and Reunion** Coordination-Allow institutions to plan, promote, and manage their alumni meet-ups, reunions, and webinars through the platform seamlessly.

- ✓ **Job and career** support-Bridge alumni with their employment opportunities by offering job listings and internships, and providing mentorship programs.

- ✓ **Membership and payment integration**-This makes subscription, event fee, and donation payments seamless and secure via PayU or Razorpay.

- ✓ **News and achievements** showcase-A place for institutions to publicize alumni success heights, major updates, or various achievements.

- ✓ **Data Driven Insights**-Use analytics to track alumni engagement trends, rates, and participation in applications to help institutions make better-informed decisions for the growth of the community.

1.3 Problem Statement :-

The partnership between alumni and their other institutions is, hereby, challenged because of lack of established communication, centralized data management, and avenues for engagement. Most institutions face the challenge of keeping track of alumni, thus losing on opportunities of networking, mentorship, career advancement, and corporative assistance given by the institution. Old methods of managing alumni manually-record-keeping or perhaps setting up groups on any social media platform are inefficient, disorganized, and devoid of any helpful features that could drive long term engagement.

Alumni also suffer from becoming informed about events, contributing to, or even interacting with their fellow graduates. They also lack a platform for finding these mentors and career advisers on which experienced alumni could assist students and much younger graduates with such a structured arrangement. In addition, the absence of an integrated payment system complicates the effective management of membership fees, event registration, and donation processes.

This Alumni Web App is designed to ameliorate all of these issues by providing a centralized digital platform for networking, events, mentorship, and institutional support for alumni. In terms of technological advancements, the solution thus provides a new avenue for alumni engagement and professional relationships and thereby developing that long-term connection of graduates with the institution.

Proposed Solution :-

The Alumni Web App provides a centralized digital platform for improving alumni engagement, communication, and institutional ties. Secure and private authentication mechanisms are there for both alumni and administrators to maintain profile accessibility and networking. The alumni database can be efficiently organized, engagement analyzed, and events such as reunions and meet-ups planned through an interactive dashboard for the administrator. A news and announcements module will provide alumni with information about developments, achievements, and upcoming opportunities within the institution. It will also include a job board or a place where the alumni will find posting of jobs, career advice, and mentoring students. The payment gateways like PayU or Razorpay will allow transactions for membership, registrations to events, and donations. Well meaningful and

value-added activity with technology will enhance the value of this Alumni Web App in improving the relationship between alumni and the institution, thus promoting lifelong professional development.

Alumni Web App is an online platform meant for centralizing alumni engagement, smoothing communication, and weaving strength into institutional ties. Both alumni and administrators are given secured, private authentication access for easy profile accessibility and networking. An administrator can manage the whole alumni database efficiently, analyze engagements, and plan occasions like reunions and meetups via an interactive dashboard. A news and announcements module will provide current developments in the institution, as well as success stories and upcoming opportunities for the alumni. An employment board mentoring students, and the sharing of job opportunities and career advice will also be available. Payments through membership, event registration, and donations are made possible through integrated payment gateways such as PayU and Razorpay. This is a meaningful and valuable engagement through technology, and hence, the value of this Alumni Web App is improved in strengthening the relationship between the alumni and the institution and encouraging lifelong professional development.

1.4 System Model of the Student Simplified Surface App :-

The System Model of the Alumni Web App shows how key modules flow data while including human interactions with the system. It provides an overview of the platform's internal operations at a higher level and specifies relationships between different entities such as alumni, administrators, and the backend server.

The system works based on three-dimensional layers, which are the Frontend (Client Side), Backend (Server Side), and Database Layer. The frontend interface is where both alumni and administrators interact and manages profiles, events registration, and communicates within the network. Employing either BootStrap or Tailwind CSS to style the web app will facilitate the interaction. The middle layer of the backend does provide user request processing, it also handles authentication via Spring Security and JWT, and manages business logic, which includes maintaining records of alumni engagement, event management, and mentorship coordination. The database layer does store structured data, like alumni profiles, event information, payments, and analytics, for efficient retrieval and secured storage.

The setup connects with either PayU or Razorpay for smooth financial transactions with respect to membership requests, event registration, and donations. The secured notification module (SMTP/JavaMail API and push notifications) lets alumni know when important things need their attention. Analyzing alumni engagement via data analytics tools (such as Power BI or in-built reporting tools) will help admin perform better in analyzing engagement trends and improving alumni interaction.

By clearly defining the interactions between different entities, the Alumni Web App System Model provides a structured, secure, and scalable way of increasing alumni engagement and institutional collaboration.

1.4.1 User Roles and Interfaces :

User Roles and Interfaces in the Alumni Web App

The **Alumni Web App** supports multiple user roles, each with distinct functionalities and interfaces to enhance engagement and streamline administration.

1. Alumni (Registered Users)

Role: Alumni can log in, manage their profiles, connect with peers, participate in events, and contribute to discussions.

Interface Features:

- **Dashboard:** Displays alumni profile, notifications, and recent updates.
- **Profile Management:** Update personal details, career information, and achievements.
- **Networking & Mentorship:** Connect with other alumni, find mentors, and offer guidance.
- **Events & Reunions:** View and register for alumni events and reunions.
- **Job Board:** Explore and share job opportunities within the alumni network.
- **Payments & Donations:** Pay membership fees, register for events, and contribute donations via **PayU/Razorpay**.

- **News & Announcements:** Stay informed about institutional updates and alumni success stories.

2. Administrators (College Admins & Moderators)

Role: Manage alumni data, oversee events, handle financial transactions, and moderate community interactions.

Interface Features:

- **Admin Dashboard:** Overview of alumni statistics, event participation, and engagement metrics.
- **Alumni Management:** View and categorize alumni data (batch-wise, region-wise, etc.).
- **Event Management:** Create, update, and track events and reunions.
- **Payment & Membership Management:** Handle membership subscriptions and donations via **PayU/Razorpay**.
- **News & Content Moderation:** Publish alumni news, success stories, and announcements.
- **Analytics & Reports:** Generate insights using **Power BI or in-app analytics** to track engagement.

1.4.2 System Architecture :-

The **Alumni Web App** follows a **three-tier architecture**, ensuring scalability, security, and efficiency in alumni engagement. The system is structured into **three primary layers**:

1. Presentation Layer (Frontend) – User Interface

This layer provides an interactive and responsive **user interface** for alumni, administrators, and visitors.

- **Technologies Used:** BootStrap , Tailwind CSS.
- **Features:** Profile management, event participation, job board, alumni networking, payments, and notifications.
- **Communication:** Sends requests to the backend via **RESTful APIs**.

2. Business Logic Layer (Backend) – Application Server

This layer is responsible for processing user requests, enforcing business rules, and managing data interactions.

- **Framework:** Spring Boot (Java) with RESTful APIs.
- **Security:** Spring Security with JWT for authentication and role-based access control.
- **Functionalities:**
    - Alumni and admin management
    - Event and news handling
    - Payment processing (PayU/Razorpay)
    - Notifications (Email & push)
    - Data analytics and reporting

3. Data Layer (Database) – Storage and Management

This layer handles all structured and transactional data.

- **Database:** MySQL with Hibernate ORM.
- **Stored Data:** Alumni profiles, event details, transactions, messages, job postings, and analytics logs.
- **Backup & Security:** Regular database backups and access control to ensure data integrity.

4. External Integrations

The system connects with external services for seamless operations.

- **Payment Gateway:** PayU/Razorpay for secure transactions.
- **Email Services:** SMTP/JavaMail API for notifications.
- **Analytics & Reporting:** Power BI or built-in reporting for tracking alumni engagement.

Data Flow in the System

1. **User Request:** Alumni/Admin interacts with the frontend.
2. **API Call:** Requests are sent to the backend for processing.
3. **Business Logic Execution:** The backend validates and processes the request.
4. **Database Interaction:** Fetch/store required data.
5. **Response to Frontend:** Processed data is sent back to the UI.
6. **External API Calls:** If needed (e.g., payments, email notifications).

1.5 Project Time Line Diagram (Gant Chart) :

Task	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W 10	W 11	W 12	W 13	W 14	W 15
Planning & Research															
System Design & Architecture															
UI/UX Design															
Alumni Interface Development															
Admin Interface Development															
Backend System Setup															
API Development															
Testing															
Final Deployment & Launch															

CHAPTER – II

2.1 Module Identification and Description of Student Simplified Surface App :-

The Alumni Web App encompasses several core modules that promote alumni engagement, streamline administrative processes, and ease communication between alumni and the institution. This way, the modules interconnect and create a participatory platform for both alumni and administrators. Below is a comprehensive identification and description of the main modules in the app.

User Authentication & Role Management Module :-

The responsibility of the UAM (User Authentication Module) consists largely in registration, authentication, and access control based on role for alumni, administrators, and super admins. The UAM is there to ensure that login is secure and user experiences are differentiated according to roles.

User Registration:

The alumni and administrators provide their basic details such as name, email, and graduation year.

An Alumni ID will be created uniquely based on the rules described to ensure structured identification.

User Login:

Users will have to log in using email/Alumni ID and a password secured by Spring Security & JWT authentication.

Role Assignment & Access Control:

Users are classified into Alumni and Admin each having varied permission layers and dashboard views.

Administrators manage alumni data, events, and content are responsible for overall system configurations.

Alumni Dashboard Module :-

This module is basically the single interface for alumni. It has a user-friendly dashboard that allows access to different features pertaining to academics like adding, viewing news and stories, adding Job Application, and checking as well as updating the profile.

Add Stories and News	Allow Alumni to view and add news and stories
Add Job Application	Alumni can upload any type of Job application , So the other alumni can view and apply the job for the given role.
View and Edit Profile	The Logged in Alumni Can View their profile and edit their any field in his profile
Check Alumni Directory	Allow to see how many alumni members are available and be a part of the institution.
Life Membership Purchase	Alumni can pay 1500 rupees to be an part of an Life Member in Alumni website.
Contact And Send Message	When Alumni required any Documents or any queries they can send the message in contact section the admin will check and reply in email.

Admin Dashboard Module :-

This module caters to the admin providing access to tools for Upload News and Stories, Manage Alumni Details, Upload Existing Alumni details or Membership details in xlsx or csv format, Check Message and Reply. It allows admin to monitor alumni activity and update institutional data efficiently.

Upload News and Stories	Admin can Add , Update and Delete the News and Stories that was uploaded
Manage Alumni Details	Admin can check alumni details year wise and they can export the data in to .xlsx format
Upload Existing Alumni details or Membership details in xlsx or csv format	When admin have the existing alumni details or membership details .xlsx or .csv can upload in to database and check the details.
Check Message and Reply	Check the alumni sender message and send the

	reply in alumni email

News and Story Module :-

The News and Management Module facilitates the process of alumni can easily add the news and stories that they are experienced in their life and expressed in alumni website

Alumni News	Alumni will be able to add and view the news also they can share in social media channels.
Alumni Story	Alumni can add and view the story same as they can share the story in social media channels

Job Application Module :-

This module focuses on managing , viewing and uploading an Job application to the alumni website so any alumni can be easily view and apply the job for the selected role.

Job Apply Upload (Alumni)	Allows alumni to upload the different and various job offer's
View and Apply (Alumni)	Alumni should be easily view and apply the job to move their career in different organization

Life Membership Module :-

This module provides a centralized platform for alumni to purchase the alumni life membership if they are already and Life Member they can check with their phone number

Enrol Membership (Alumni)	Alumni can enrol or purchase the life membership with paying of 1500 rupees
Verify Membership (Alumni)	When alumni already have the membership they can verify with phone number
Pdf Generator	After completing the purchase it automatically generate the

	pdf for the alumni

2.2 User Roles and Characteristics of Alumni Web App :-

The **Alumni Web App** has two primary user roles: **Alumni's** and **Admin**. Each role comes with its own set of characteristics, **permissions**, and **functionalities** that are tailored to meet the specific needs of the users. The app's design ensures that each user has access to the features most relevant to their role, creating a seamless experience for both alumni's and administrators.

Alumni Role :-

Primary User	Alumni's are the main users of the app, and their role is focused on accessing instituion information and services.
Limited Access	Alumni's have restricted access to features specific to their needs, such as viewing alumni contact details , Admin details.
Interactive Interface	The alumni interface is designed to be user-friendly and easy to navigate, with a focus on simplicity and functionality.
Add News and Story	Alumni's can add and view the news and stories
Generate Membership (PDF)	After the Membership purchase it will generate the PDF for the desired alumni
Purchase Membership	Alumni can purchase the life membership without any offline or any paper work they will make the payment in online.
Check Alumni Details	Alumni can check how many alumni are enrol and find with their name and department
Send Message	When Alumni have any queries or need to update

	anything they can send the message to admin , The admin will check and send the response in alumni mail

Admin Role :-

Secondary User	Admin have more administrative responsibilities and use the app to manage alumni-related activities.
Enhanced Access	Admin users have access to more features than alumni's, including managing every data access.
Efficient Management Interface	The admin interface is designed to be comprehensive but simple, allowing them to efficiently manage alumni records and activities.
Manage Uploaded News and Story	Admins can upload and manage the news and story. This helps track alumni progress.
Review and Response/Skip the Alumni Messages	Admins can view all alumni messages and they can response when the message is required otherwise they can skip.
Upload Existing Alumni or Membership Data.	When the Admin have offline alumni details and membership details in .xlsx or .csv format they can upload it will stored in database.
Check and Export Alumni Details	Admin can check and export the alumni details in .xlsx format
Add and View User Details	Admins can monitor how many user are added and check their details

2.3 Functional Requirements for the Alumni Web App :-

1. **User Authentication:**
    - The system must allow alumni and admin to register and log in.
    - Admins must be able to differentiate between roles (alumni and admin).
2. **Alumni Dashboard:**
    - Alumni's should be able to add and view the news and story.
    - Alumni's should be able to add and view Job application and apply the desired job for their roles.
    - Alumni's should have access to check and edit their profile in any field.
    - The dashboard must display stories and job applications.
3. **News and Story :**
    - Alumni's must be able to add and view the various and different kinds of news and stories.
    - The system should display a table layout showing how news and stories are uploaded
    - Admins should be able to review and delete the old news and stories.
    - Admins must be able to check and added new stories and news with their side.
4. **Job Application:**

- The system should display the uploaded and newly added job sectors for alumni's.
- Alumni's should be able to view and apply the job's anytime they want.

5. **Message Section:**
    - Admins must be able to check and response/skip the alumni's messages.
    - The message will be send the response to the alumni's email.
    - Alumni's should receive notifications in her email for their queries.
6. **Membership Purchase:**
    - Admins should be able check and download the alumni membership details in pdf format.
    - Alumni's must be able to purchase the membership after the registration with their full name and phone number.
    - The app should categorize membership based on alumni year of purchase.
7. **Upload System:**
    - The app must send push the offline alumni details and membership details to database.
    - After the upload it will check duplicate values and remove the duplicate values.
8. **User Role Management:**
    - The system must assign different roles to students and admin.
    - Each role should have access only to features relevant to their responsibilities (e.g., alumni's cannot delete news and stories, admins cannot apply for any job application).

2.4 Performance Requirements for the Alumni Web App

1. **Response Time:**
    - The app should load the dashboard within 2-3 seconds after login.
    - The system must process alumni and admin applications within 2 seconds and reflect them in the desired layout.
    - Every details are fetch and display in unique format with in 1-0.5 millisecond.
2. **Scalability:**
    - The app should be able to handle concurrent use by at least 5000 alumni's and 10 admin without performance degradation.
    - It should efficiently handle large volumes of data (e.g., register of alumni , adding news and stories) without slowing down.
3. **Data Consistency:**
    - All data (news, stories and job applications) should be consistently updated and available in real-time to both alumni's and admins.
    - Any changes made by the admin (e.g., responding message, adding news) must reflect immediately on the alumni's dashboard.

4. **Availability:**
    - The app should maintain 99.9% uptime to ensure alumni's and admin have access to the system when needed.
    - The notification system must always be active and send alerts in real-time.
5. **Security:**
    - Although passwords are not encrypted, the app must ensure secure communication between users and the system (e.g., using HTTPS).
    - Data must be protected from unauthorized access, and role-based permissions should be strictly enforced.
6. **Error Handling:**
    - The app must display appropriate error messages for invalid actions (e.g., incorrect login details).
    - System errors should be logged and monitored to ensure timely fixes and updates.
7. **Memory Usage:**
    - The app should be optimized to use minimal device memory and not exceed 100MB of RAM during normal operation.

2.5 Hardware and Software details (Implementation and Deployment Side) :-

Implementation Side :-

Hardware Requirements :

1. **Development Machine:**
    - Processor: Intel Core i3 or equivalent (Minimum), Intel Core i5 or better (Recommended)
    - RAM: 4 GB (Minimum), 16 GB or more (Recommended for running multiple tools simultaneously)

2. **Mobile Device(s):**
    - Android smartphones (preferably running Android 8.0 and above) for testing the app on physical devices.
    - USB cables and ports for connecting devices to the development machine for debugging.

Software Requirements :-

1. **Operating System:**
    - Windows 10/11 (Preferred for wider compatibility with development tools)
2. **Development Tools:**
    - Spring Boot Tool Suite: Primary IDE for building the Web app, including Java programming, UI/UX design, and debugging.
    - Java Development Kit (JDK): For compiling and running Java code used in the app.
3. **Database Management:**
    - MySQL : For managing and storing alumni data (e.g., alumni info, news, job applications) during development.
4. **APIs and Libraries:**
    - PDF Library: A Java/Android library such as iText or PDFBox for generating membership in PDF format.
5. **Testing Tools:**
    - Postman: For testing API endpoints used in the app.

Deployment Side :-

Hardware Requirements :

1. **Cloud/Physical Server:**
    - Processor: 2+ Core CPU (Minimum) or 4+ Core CPU (Recommended for larger user bases).

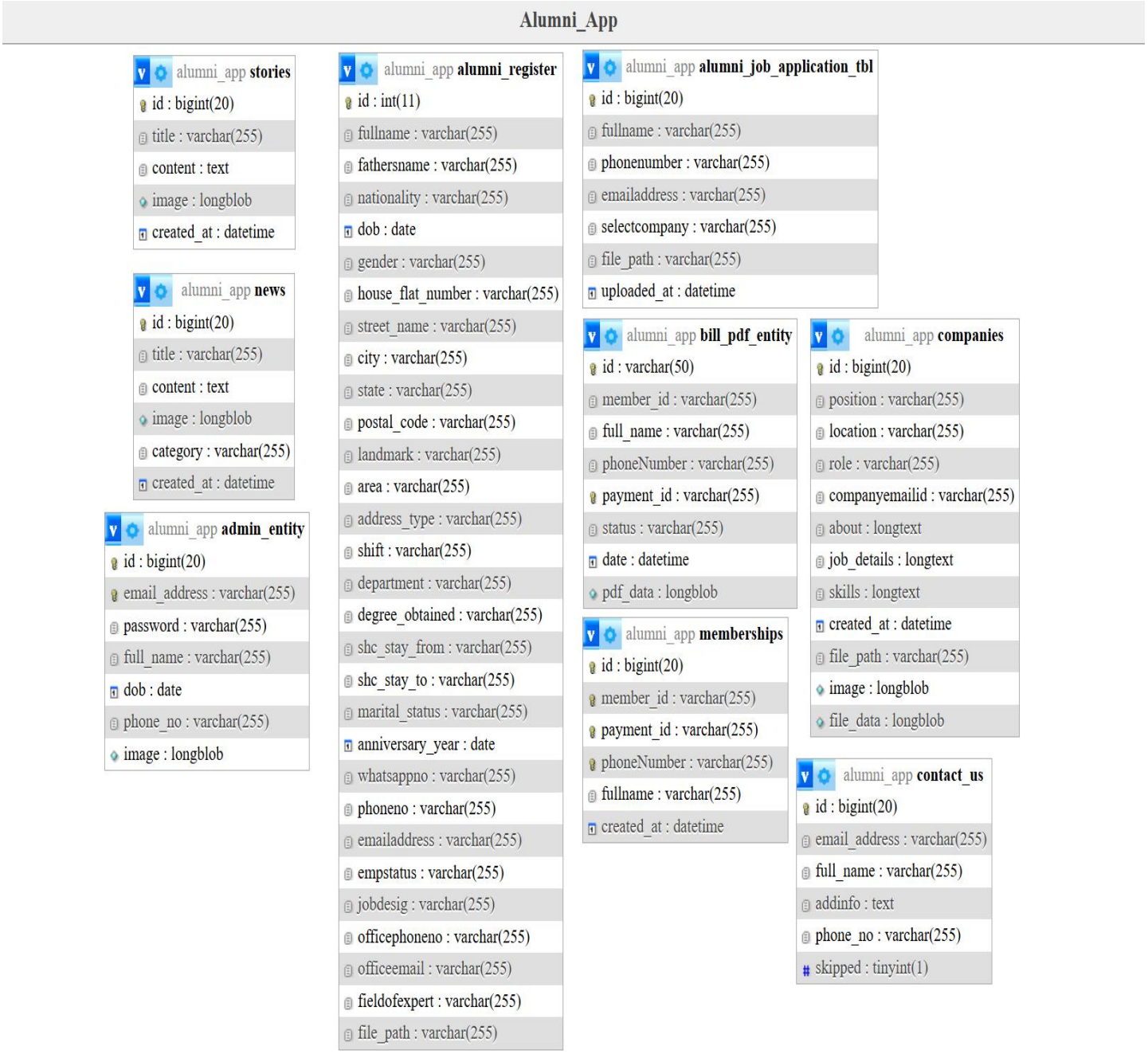
- RAM: 4 GB (Minimum), 8 GB or more (Recommended for handling large data and traffic).
- Storage: 100 GB (Minimum) SSD for faster data access (scale as per the number of users and data retention needs).
- Bandwidth: Scalable bandwidth to handle traffic spikes, especially during registration or result updates.

2. **Mobile Devices:**
    - The app will be deployed on Android smartphones running Android 8.0 (Oreo) and above.
    - Students and staff/admin will use these devices to access the app via the Google Play Store or directly through APK installation.

Software Requirements :

1. **Operating System (Server):**
    - Linux (Ubuntu) or CentOS: Commonly used for web servers due to stability and efficiency.
    - Windows Server: Can be used if the app requires a more Windows-centric server environment.
2. **Web Server:**
    - Apache: To host the backend services, API endpoints, and serve data to the Android app.
3. **Database:**
    - MySQL: Relational database system used for storing user information.
4. **Backend Technologies:**
    - Java: For building server-side logic, API integration, and managing database interactions with the app.
    - MySQl: For more scalable and real-time backend services.

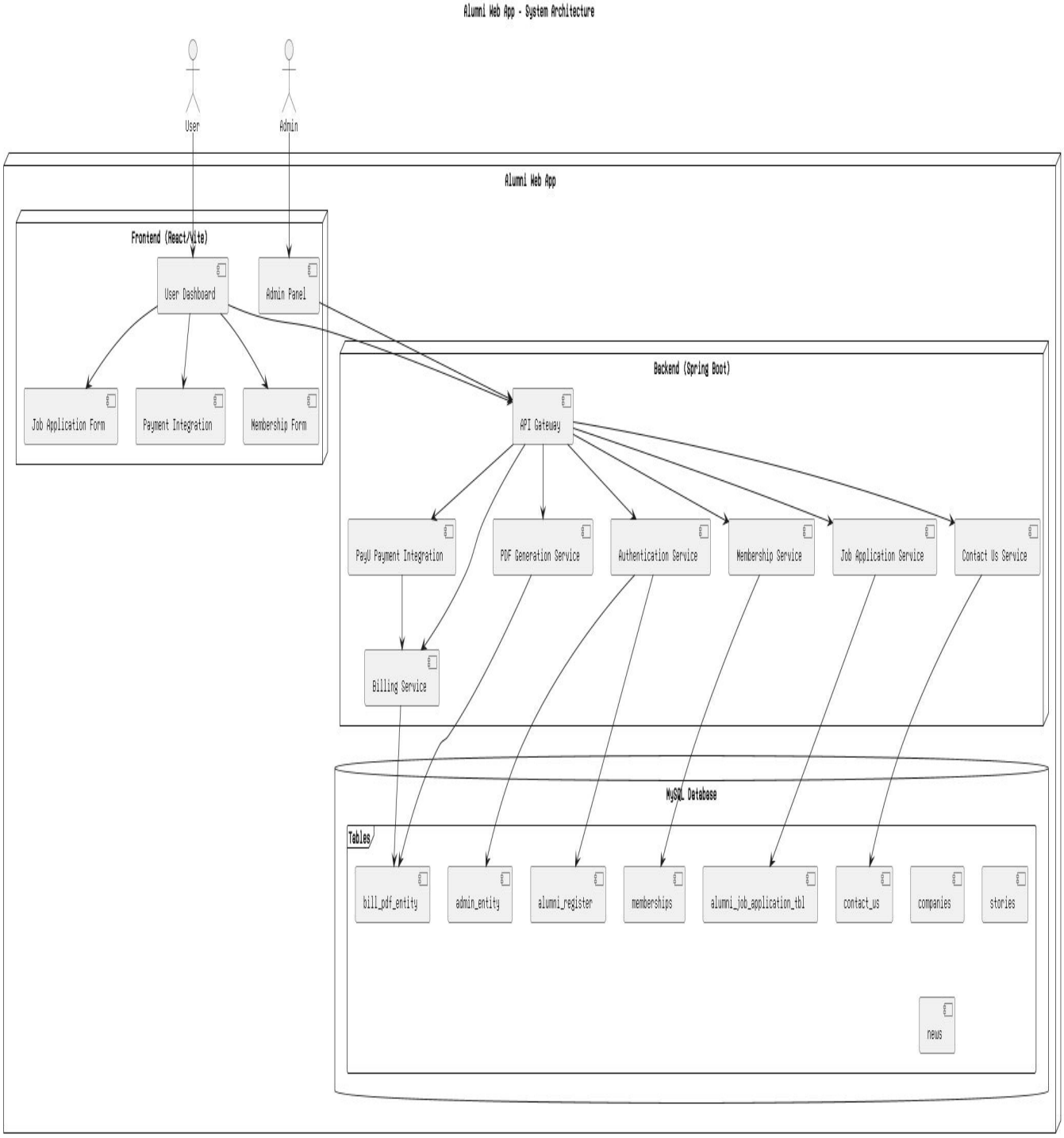
2.6 Conceptual Diagram (DB entity relationship diagram) :-



CHAPTER – III

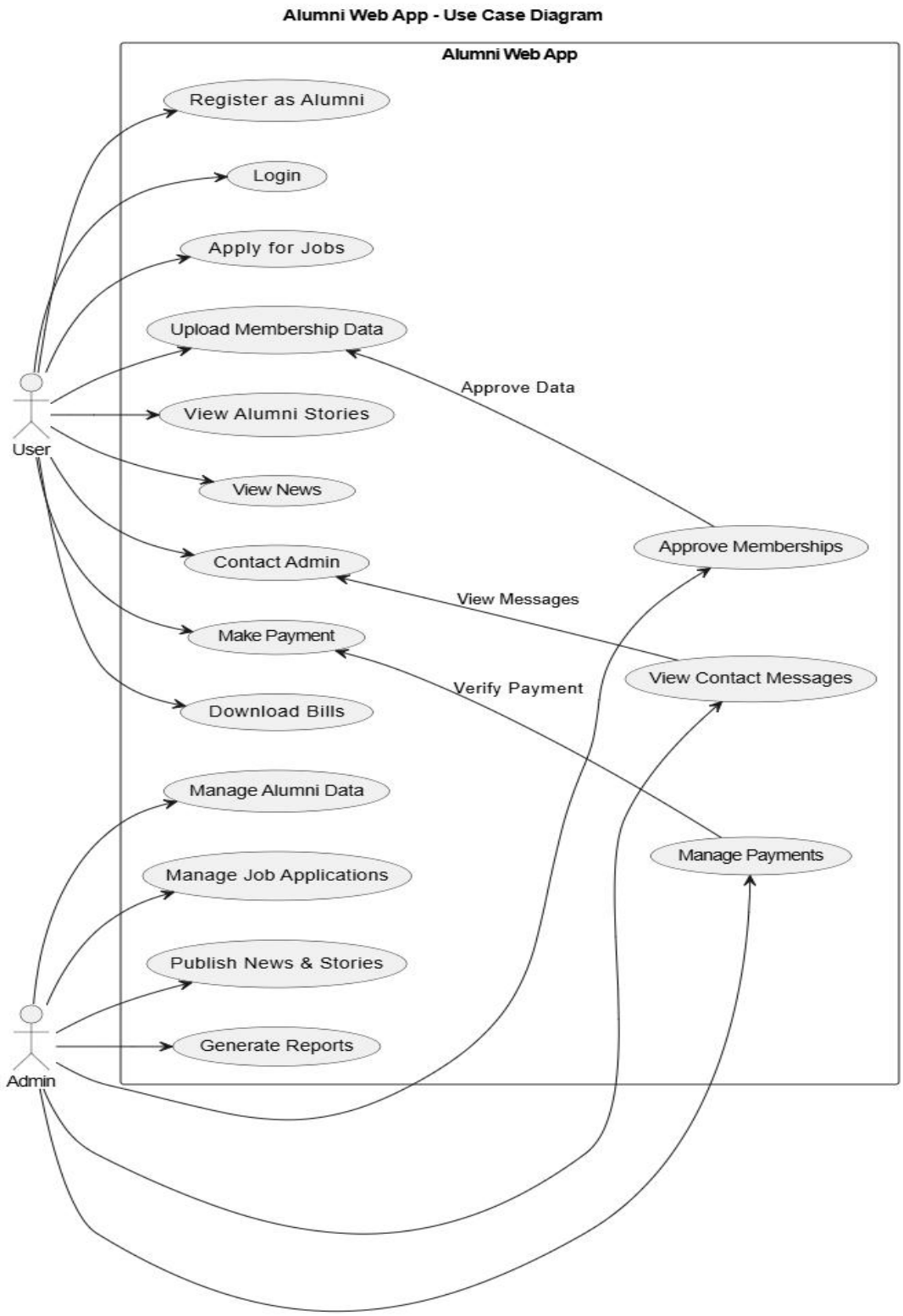
DESIGN SPECIFICATIONS

3.1 System Architecture Diagram

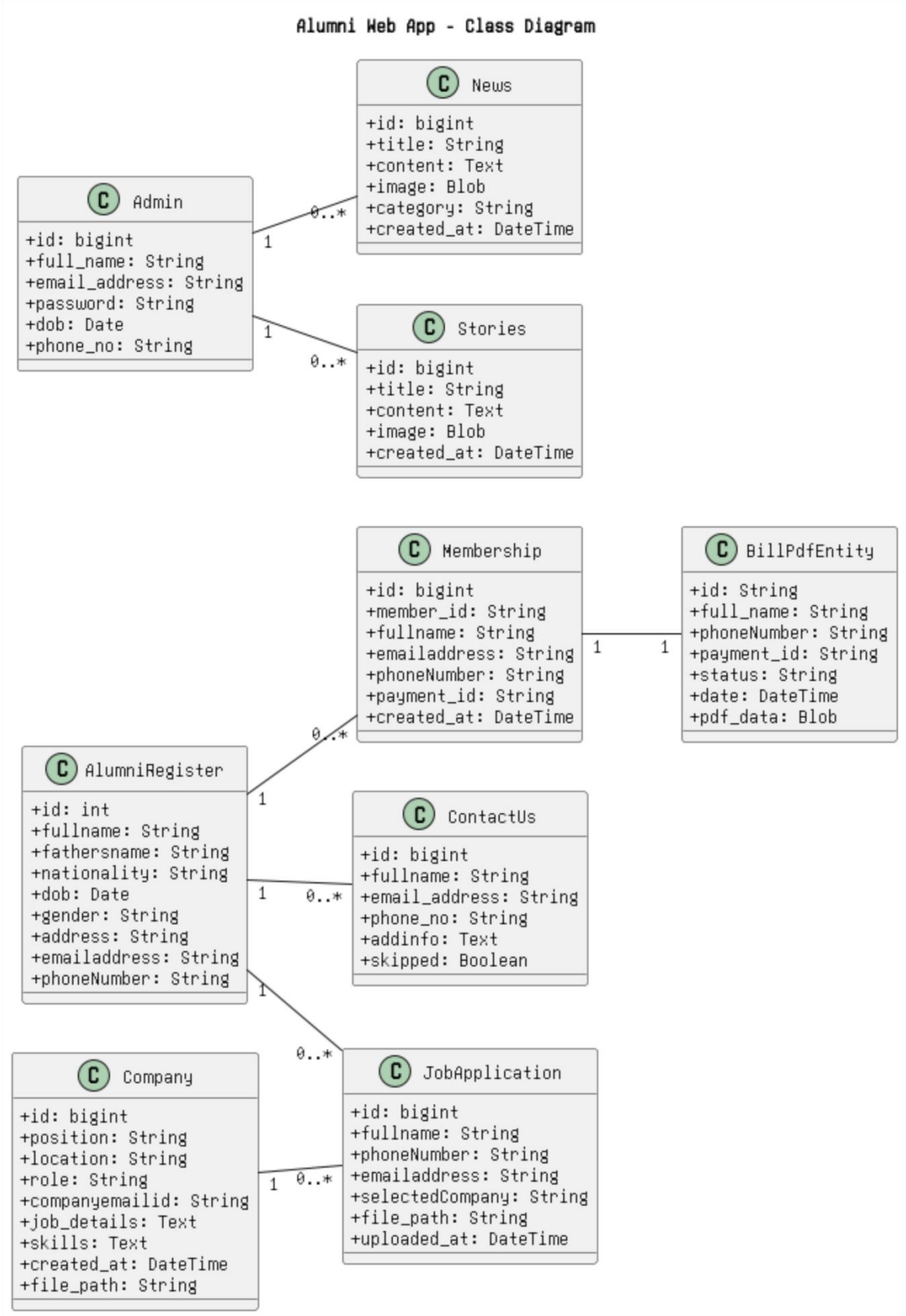


3.2 UML Design Models

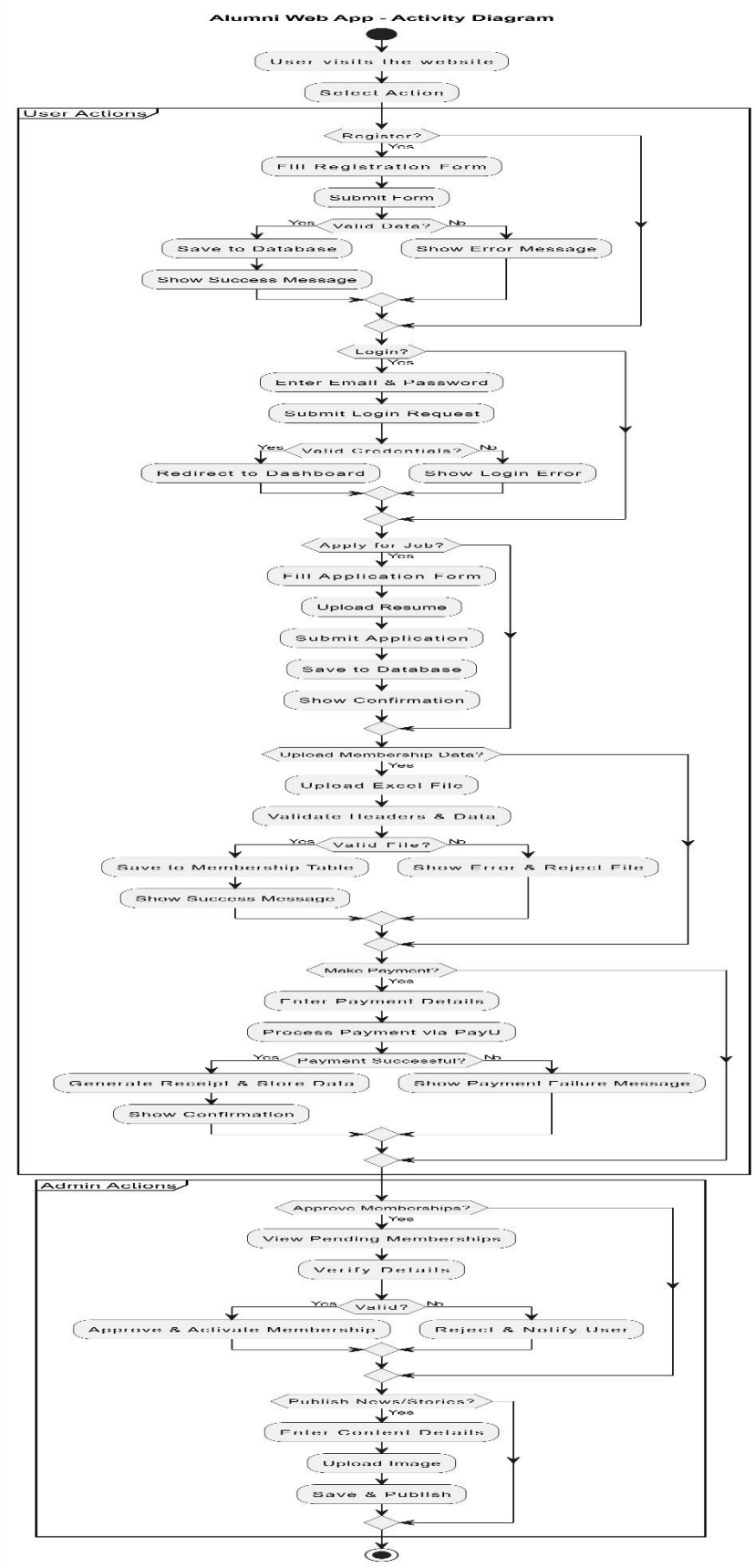
Use Case Diagram :



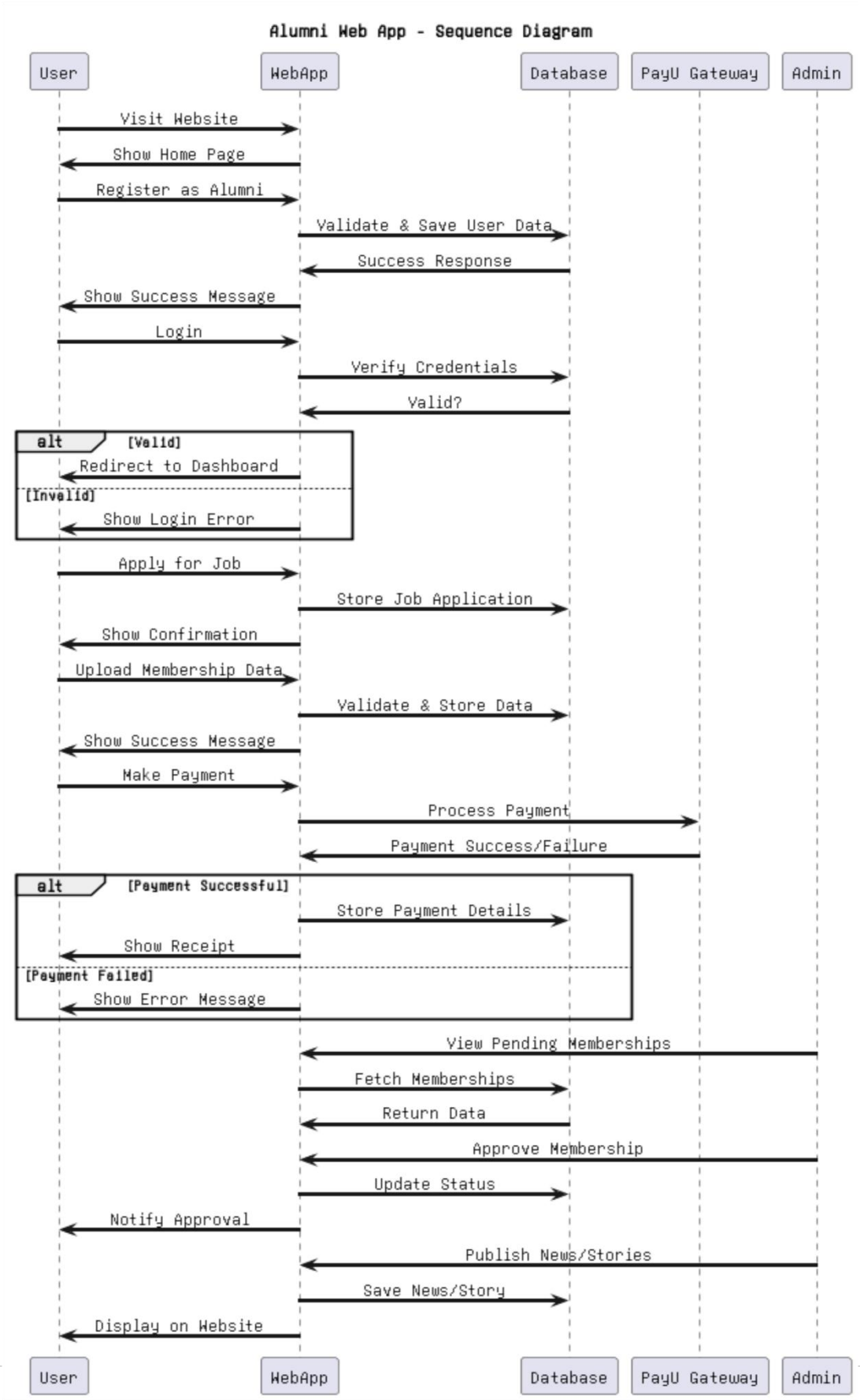
Class Diagram :



Activity Diagram :



Sequence Diagram : -



3.3 Database Design

Table Structure

S.No	TABLE NAME	DESCRIPTION
1	**Alumni Register And Login**	The Alumni's can Register Form will be available they need to enter and submit every values for successful registration process.
2	**Admin Register And Login**	The Admin can Register with limited data and use the same data for Login as well.
3	**News Table**	The Alumni Can Add and View the News , Only the Admin have access to Add , Delete and view.
4	**Stories Table**	The Alumni have only access to add and view the stories , But Admin can add stories and delete the stories.
5	**Memberships Table**	When the alumni registration is success it move to membership page to purchase the membership if they are already an member they can verify else they can enrol membership.
6	**Companies Table**	The Alumni can upload or add the job's using add new job button it will added and displayed in alumni job list page.
7	**Alumni Job Application Table**	This will store how many alumni are applied the various jobs and which company they are applied.
8	**Contact Us Table**	It will stores alumni message in to database and displays in admin section so the admin can view and response it to email.
9	**Bill Pdf Table**	After the Membership online payment is done It will generate the member id and pdf for the alumni.

Alumni Register And Login:

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment
Full Name	Varchar	255	Not Null	Name of the Alumni
Fathers Name	Varchar	255	Not Null	Name of the Alumni father
Nationality	Varchar	255	Not Null	Nationality of the Alumni
Dob	Date	-	Null	Date of Birth of the Alumni
Gender	Varchar	255	Not Null	Alumni Gender Male or Female
House Flat Number	Varchar	255	Not Null	House Number of Alumni
Street Name	Varchar	255	Not Null	Street Name of Alumni
City	Varchar	255	Not Null	City of Alumni
State	Varchar	255	Not Null	State of Alumni
Postal Code	Varchar	255	Not Null	Postal code of Alumni
Landmark	Varchar	255	Not Null	Landmark of Alumni
Area	Varchar	255	Not Null	Area of Alumni
Address Type	Varchar	255	Not Null	Address Type permanent or current
Shift	Varchar	255	Not Null	Shift of Alumni 1 or 2
Department	Varchar	255	Not Null	Department of Alumni
Degree Obtained	Varchar	255	Not Null	How Many Degree's
SHC Stay From	Varchar	255	Not Null	Alumni college from year
SHC Stay To	Varchar	255	Not Null	Alumni college to year
Marital Status	Varchar	255	Not Null	Alumni marital status yes or no
Anniversary year	Date	-	Not Null	Alumni anniversary year
Whatsappno	Varchar	255	Not Null	Alumni Whatsapp No
Phoneno	Varchar	255	Not Null	Alumni Phone no

Emailaddress	Varchar	255	Not Null	Alumni Email Address
Emp Status	Varchar	255	Not Null	Alumni Employee Status
Job Desig	Varchar	255	Not Null	Alumni Job Designation
Office Phone No	Varchar	255	Not Null	Alumni Office Phone no
Office Email	Varchar	255	Not Null	Alumni Office E-mail
Field of Expert	Varchar	255	Not Null	Alumni field of Expert
File_path	Varchar	255	Not Null	Stored image path

Admin Register And Login :

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
Email Address	Varchar	50	Not Null	Admin Email Address
Password	Varchar	50	Not Null	Admin Password
Full Name	Varchar	50	Not Null	Admin Full Name
Dob	Date	50	Not Null	Admin Date of Birth
Phone No	Varchar	50	Not Null	Admin Phone no
Image	longblob	-	Not Null	Admin Image

News Table:

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
Title	Varchar	255	Not Null	News Title
Content	Text	255	Not Null	News Content
image	longblob	-	Not Null	News Image
category	Varchar	255	Not Null	News Category
created_at	datetime	-	Not Null	News Stored Date

Stories Table :

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
Title	Varchar	255	Null	Story Title
Content	Text	-	Null	Story Content
Image	Longlob	-	Null	Story Image
Created at	Timestamp	-	Not Null	Story Created Date

Memberships Table :

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
Member id	Varchar	11	Null	Auto Generating Id for Payments
Payment Id	Varchar	100	Null	Saving Payment Id
Full name	Varchar	255	Null	Saving the Fullname of the Alumni
Phone number	Varchar	255	Null	Saving if the Phone number of the Alumni
Created_at	Timestamp	-	Not Null	Creating current Time Stamp

Companies Table :

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
position	Varchar	50	Not Null	Adding Position
location	Varchar	255	Not Null	Saving Location
role	Varchar	255	Null	Saving the Role
companyemailid	Varchar	255	Null	Saving the company email address
about	Varchar	255	Null	Saving the Company About

job_details	Varchar	11	Not Null	Saving the company job details
skills	Varchar	255		Saving company skills
Created_at	Timestamp	-	Not Null	Creating current Time Stamp
file_path	Varchar	255		Saving uploaded file path
image	Longlob	-		Saving company logo
file_data	Longlob	-		Saving Company uploaded files

Alumni Job Application Table:

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
fullname	Varchar	11	Null	Full Name of Job Applier
phonenumber	Varchar	255	Not Null	Phone No of Job Applier
emailaddress	Varchar	255	Not Null	Email Address of Job Applier
selectcompany	Varchar	255	Not Null	Selected Company
file_path	Varchar	255	Not Null	File Path

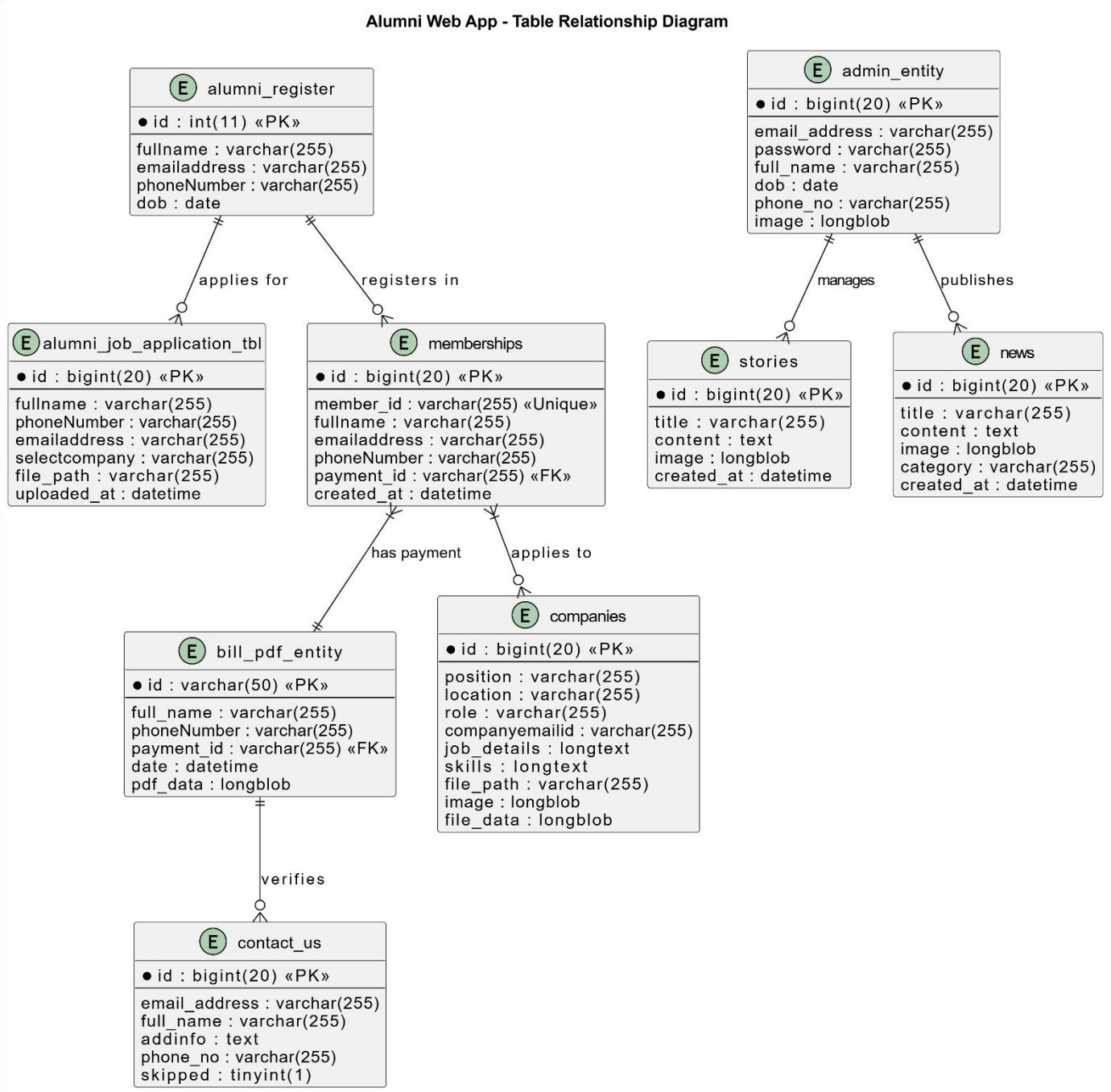
Contact Us Table:

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Increment Id
email_address	Varchar	50	Null	Storing Email Address
full_name	Varchar	11	Null	Storing Full Name
addinfo	Varchar	255	Not Null	Storing Add Info
phone_no	Varchar	255	Not Null	Storing Phone No
skipped	Tinyint	1	Not Null	-

Bill Pdf Table :

Field Name	Data Type	Length	Constraints	Description
Id	Integer	11	Not Null	Auto Generating Id
member_id	Varchar	11	Not Null	Fetching Member Id
full_name	Varchar	11	Not Null	Fetching Fullname
payment_id	Varchar	50	Not Null	Fetching Payment Id
status	Varchar	100	Not Null	Storing Status Paid or Not
date	Date	-	Not Null	Storing Date of Payment
pdf_data	Longlob	-	Not Null	Storing Pdf Data
phone_number	Varchar	11	Not Null	Fetching Phone No

Table Relationship Diagram :



Query Design :

Query to Insert the Data in to News Table in MYSQL Database Table -

```
@PostMapping("/addnews")
public ResponseEntity<Object> addNews(
@RequestParam("title") String title,
@RequestParam("content") String content,
@RequestParam("category") String category,
@RequestParam("image") MultipartFile imageFile) {
try {
newsService.saveNews(title, content, category, imageFile);
return ResponseEntity.status(HttpStatus.CREATED).body("{\"message\": \"News added successfully!\"}");
} catch (Exception e) {
e.printStackTrace();
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body("{\"error\": \"Error saving news: " + e.getMessage() + "\"}");
}
}
```

Query to Insert the Data in to Stories Table in MYSQL Database Table -

```
@PostMapping("/addstories")
public ResponseEntity<Object> addNews(
        @RequestParam("title") String title,
        @RequestParam("content") String content,
        @RequestParam("image") MultipartFile imageFile) {
    try {
        storyService.saveStory(title, content, imageFile);
        return ResponseEntity.status(HttpStatus.CREATED).body("{\"message\": \"Story added successfully!\"}");
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
```

```
            .body("{\"error\": \"Error saving story: " + e.getMessage() + "\"}");
    }
}
```

Query to Insert the Data in to Companies Table in MYSQL Database Table -

```
@PostMapping("/submitCompany")

public String submitCompany(
        @RequestParam(value = "position", required = false) String position,
        @RequestParam(value = "location", required = false) String location,
        @RequestParam(value = "role", required = false) String role,
        @RequestParam(value = "about", required = false) String about,
        @RequestParam(value = "jobdetails", required = false) String jobdetails,
        @RequestParam(value = "skills", required = false) String skills,
        @RequestParam(value = "companyemailid", required = false) String companyemailid,
        @RequestParam(value = "applications", required = false) String applications,
        @RequestParam(value = "image", required = false) MultipartFile image,
        @RequestParam(value = "fileUpload", required = false) MultipartFile fileUpload,
        Model model) {
    try {
        CompanyEntity company = new CompanyEntity()
        if (position != null && !position.isEmpty()) {
            company.setPosition(position);
        }
        if (location != null && !location.isEmpty()) {
            company.setLocation(location);
        }
        if (role != null && !role.isEmpty()) {
            company.setRole(role);
        }
        if (about != null && !about.isEmpty()) {
            company.setAbout(about);
        }
        if (companyemailid != null && !companyemailid.isEmpty()) {
```

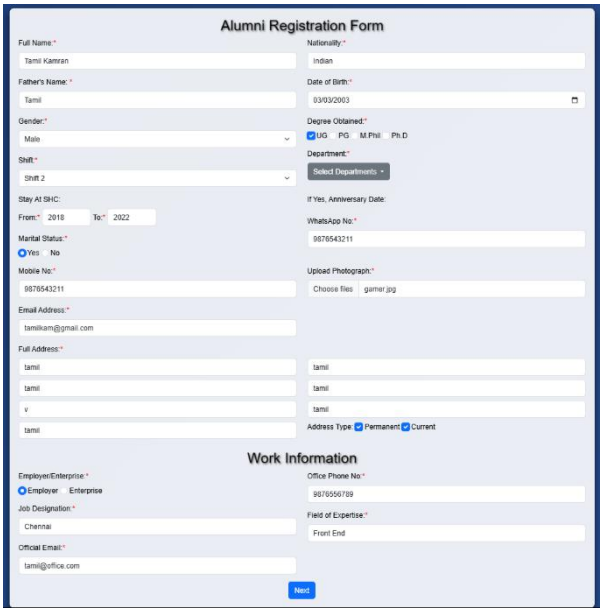
```
            company.setCompanyemailid(companyemailid);
        }
        if (skills != null && !skills.isEmpty()) {
            company.setSkills(skills);
        }
        if (jobdetails != null && !jobdetails.isEmpty()) {
            company.setJobDetails(jobdetails);
        }
        if (image != null && !image.isEmpty()) {
            company.setImage(image.getBytes());
        }
        if (fileUpload != null && !fileUpload.isEmpty()) {
            String fileName = System.currentTimeMillis() + "_" +
fileUpload.getOriginalFilename();
            Path uploadPath = Paths.get(UPLOAD_DIR + fileName);
            Files.createDirectories(uploadPath.getParent());
            Files.write(uploadPath, fileUpload.getBytes());
            company.setFileData(fileUpload.getBytes());  // Store file data
            company.setFilePath(uploadPath.toString()); // Store file path
        } else {
            company.setFilePath(""); // Avoid null value if fileUpload is empty
        }
        companyService.saveCompanyEntity(company);
        model.addAttribute("successMessage", "Company details saved successfully!");
    } catch (IOException e) {
        e.printStackTrace();
        model.addAttribute("errorMessage", "Failed to save company details. Please try
again.");
    }
    return "redirect:/home";
  }
```

Query to Insert the Data in to Contact Us Table in MYSQL Database Table -

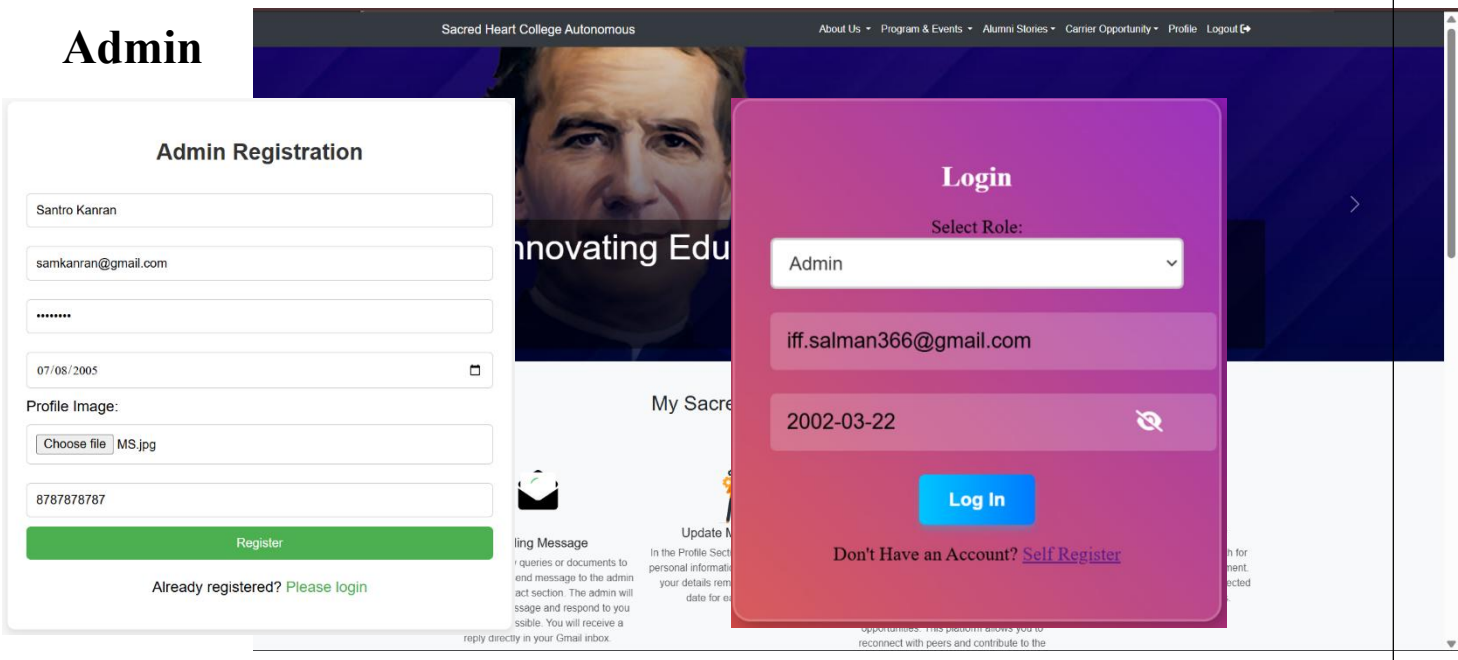
```
@PostMapping("/contactus")
public ResponseEntity<Object> addContact(@RequestBody String rawRequest) {
    System.out.println("Raw Request: " + rawRequest);
    try {
        ObjectMapper objectMapper = new ObjectMapper();
        Contact contact = objectMapper.readValue(rawRequest, Contact.class);
        System.out.println("Received fullname: " + contact.getFullName());
        System.out.println("Received phoneNo: " + contact.getPhoneNo());
        System.out.println("Received emailAddress: " + contact.getEmailAddress());
        System.out.println("Received addinfo: " + contact.getAddinfo());
        contactService.saveContact(contact.getFullName(),                contact.getPhoneNo(),
contact.getEmailAddress(), contact.getAddinfo());
        return    ResponseEntity.status(HttpStatus.CREATED).body("{\"message\":    \"Thank
you! Your message has been received!\"}");
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body("{\"error\": \"Error: Unable to submit your message. Please try again. " +
e.getMessage() + "\"}");
    }
}
```

3.4 Design for Alumni Web App :

Alumni's Dashboard :

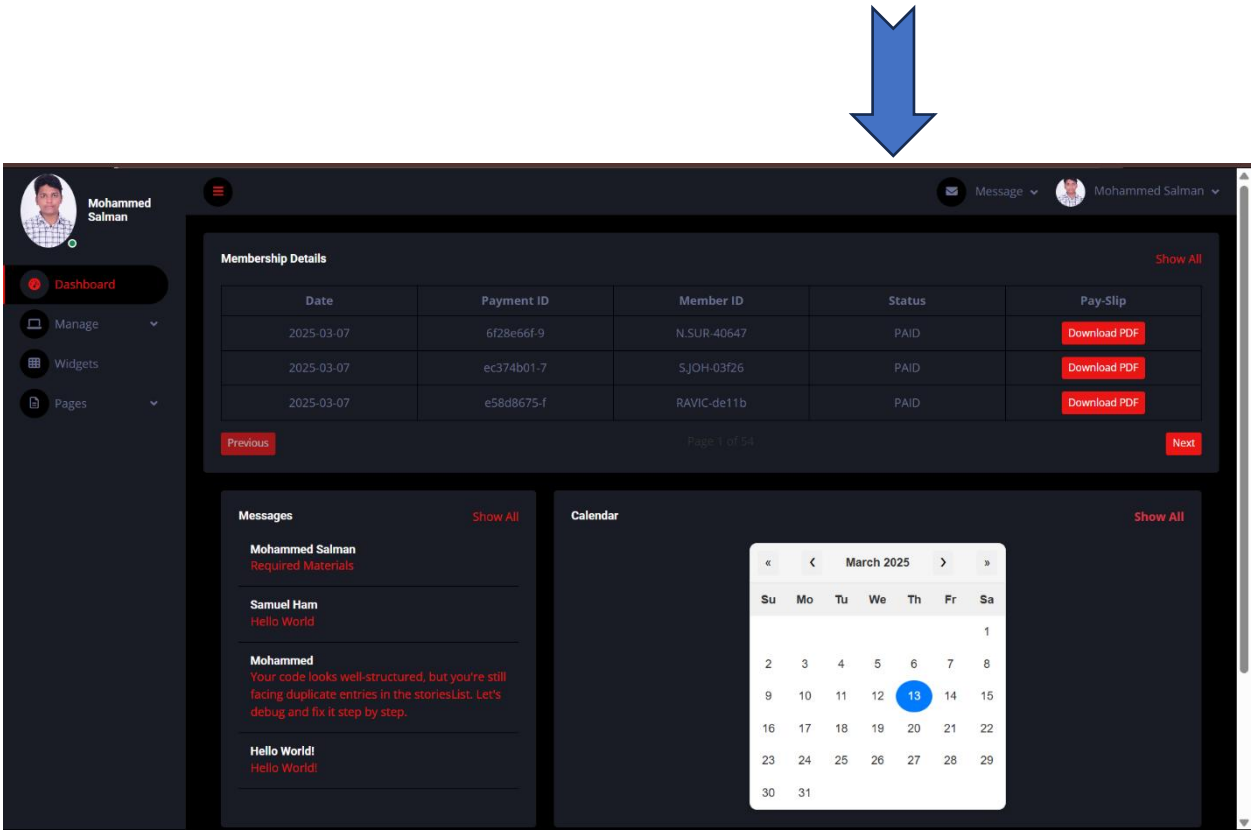


Admin



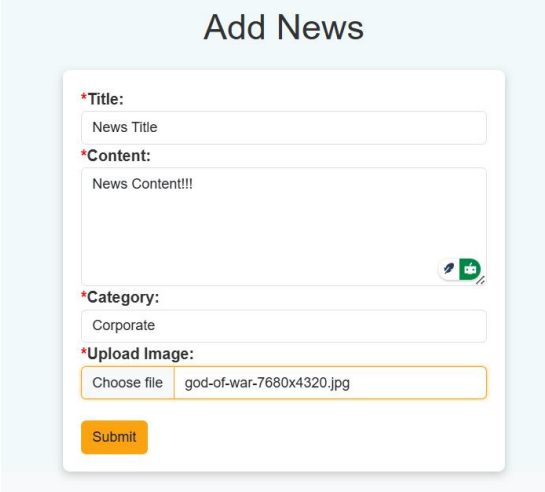
The Alumni Dashboard First, the new alumni must register their information. Once registered, they will be directed to the login page, where they can simply select the role and login with their email address or phone number and dob as a password. If the login is successful, they will be directed to the alumni dashboard, where they can perform a variety of operations.

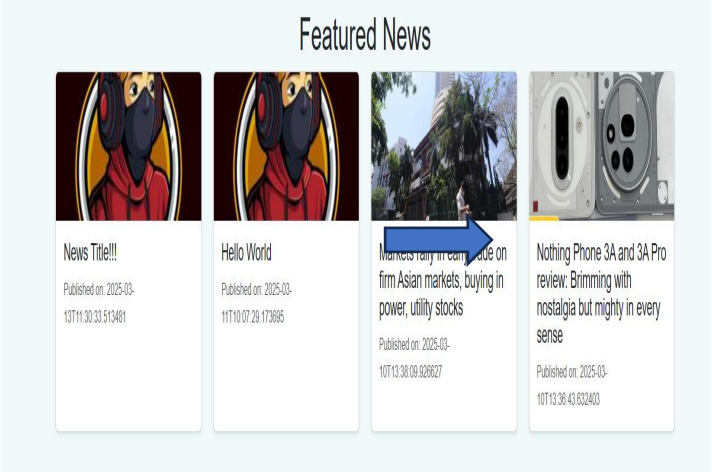
Dashboard :



In your alumni web app, the **Admin Registration and Login** process involves a secure authentication mechanism within your **Spring Boot** backend. The admin registers by providing credentials such as a username, email, and password, which are securely hashed before being stored in the database. Upon login, the credentials are verified against stored records, and a session or **JWT token** is generated for authentication. Role-based access control ensures that only authorized administrators can manage alumni records, process payments via **PayU**, and respond to user inquiries. Additionally, failed login attempts can trigger security measures like **account lockout** or **captcha verification** to enhance protection.

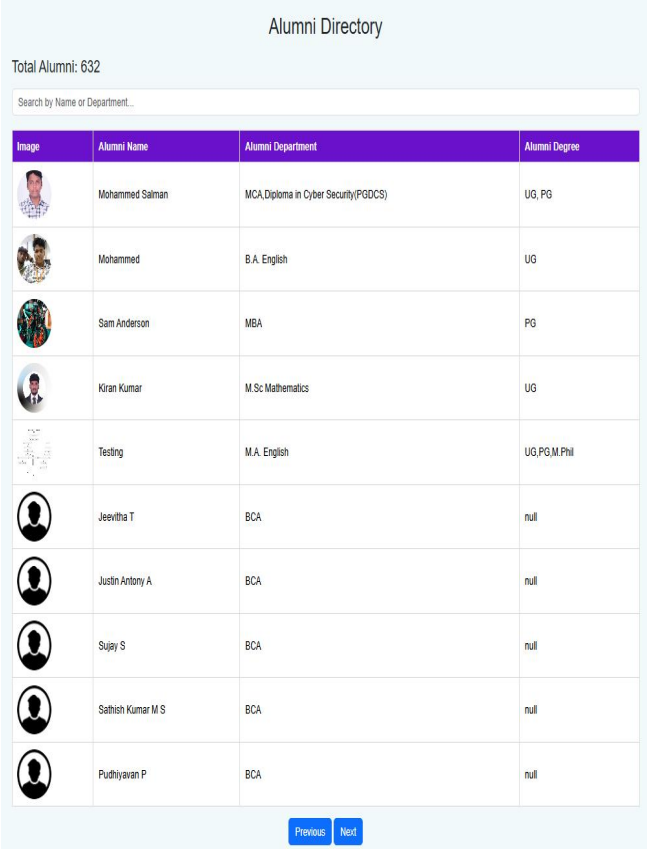
Alumni's News And Stories:





In the alumni web app, **adding and viewing news and stories** allows alumni to stay connected and engaged. Alumni can submit their **news updates and personal stories** through a form, including a title, description, images, and publication date. Once submitted, the content is reviewed and stored in the database, making it available for others to view. The **view section** displays a list of approved news and stories, sorted by date or category, with options to filter or search for specific posts. This feature fosters a sense of community, enabling alumni to share achievements, events, and experiences with their peers.

Alumni Directory and Profile:



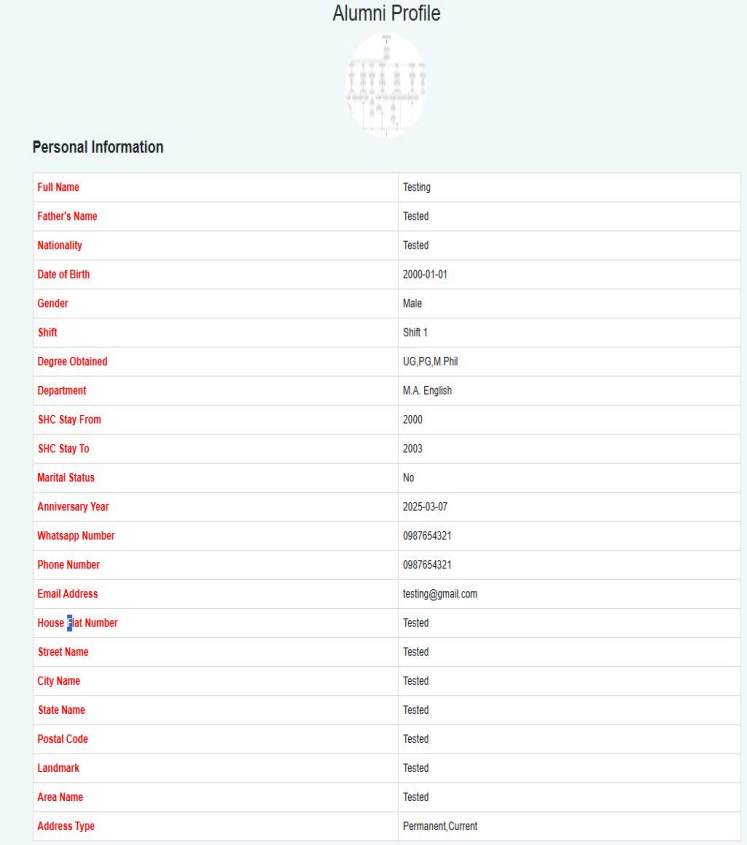
Alumni Directory

Total Alumni: 632

Search by Name or Department...

Image	Alumni Name	Alumni Department	Alumni Degree
	Mohammed Salman	MCA,Diploma in Cyber Security(PGDCS)	UG, PG
	Mohammed	B.A. English	UG
	Sam Anderson	MBA	PG
	Kiran Kumar	M.Sc Mathematics	UG
	Testing	M.A. English	UG,PG,M.Phil
	Jeevitha T	BCA	null
	Justin Antony A	BCA	null
	Sujay S	BCA	null
	Sathish Kumar M S	BCA	null
	Pudhiyavan P	BCA	null

Previous Next



Alumni Profile

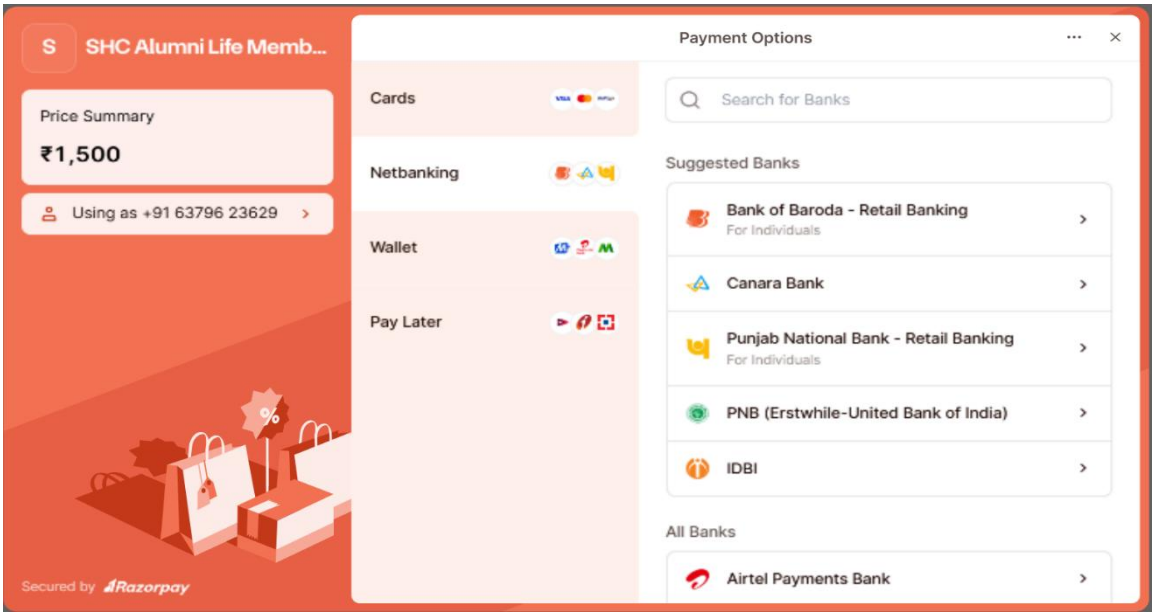
Personal Information

Full Name	Testing
Father's Name	Tested
Nationality	Tested
Date of Birth	2000-01-01
Gender	Male
Shift	Shift 1
Degree Obtained	UG,PG,M.Phil
Department	M.A. English
SHC Stay From	2000
SHC Stay To	2003
Marital Status	No
Anniversary Year	2025-03-07
Whatsapp Number	0987654321
Phone Number	0987654321
Email Address	testing@gmail.com
House Flat Number	Tested
Street Name	Tested
City Name	Tested
State Name	Tested
Postal Code	Tested
Landmark	Tested
Area Name	Tested
Address Type	Permanent,Current

In the alumni web app, the **Profile and Directory** features help alumni maintain their personal information and connect with others. Each alumnus has a **profile section** where they can update their **name, contact details, graduation year, work experience, and profile picture**. This ensures that their information remains current for networking purposes.

The **Alumni Directory** serves as a searchable database where users can find and connect with fellow alumni based on criteria like **batch, profession, or location**. To maintain privacy, certain details can be restricted based on user preferences. This feature enhances networking opportunities, helping alumni stay in touch, collaborate professionally, and support each other's careers.

Alumni Life Membership Purchase :



SACRED HEART COLLEGE (Autonomous), Tirupattur - 635 601, Tirupattur Dt.
Accredited by NAAC (5th cycle - under RAF) with a CGPA of 3.53/4 at 'A++' Grade, Afflicated to Thiruvalluvar University

Membership Bill

AUTO GENERATE PDF WHEN PURCHASE MEMBERSHIP

Full Name: Sam Anderson Kai

Phone Number: 7777777777

Payment ID: pay_Q3Xlr4XkHphsQV

PAID

Thank you for your payment!

The **Membership** feature in the alumni web app enables alumni to register for exclusive benefits and community engagement. Alumni can apply for membership by uploading their details through an **XLSX file**, which is validated and stored in the memberships table. Each member is assigned a unique **member_id**, automatically generated based on their **full name**.

Once registered, members gain access to **special events, newsletters, and networking opportunities**. Admins can manage memberships, approve applications, and generate **membership PDFs**, which are stored in the bill_pdf_entity table. This structured membership system fosters a stronger alumni network while ensuring smooth data handling and

CHAPTER – IV

Implementation of Java Code

```
package com.shc.alumni.springboot.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.shc.alumni.springboot.model.Contact;
import com.shc.alumni.springboot.service.ContactService;
@Controller
@RequestMapping("/") // Optional, adds a base path for all endpoints in this controller
public class ContactController {
    @Autowired
    private ContactService contactService;

    /
     * Endpoint to handle contact form submissions
     */
    @GetMapping("/alumni/contactus")
    public String showContactForm() {
        return "alumnicontactus"; // Returns alumniregister.jsp
    }
    @PostMapping("/alumni/contactus")
    public ResponseEntity<Object> addContact(@RequestBody String rawRequest) {
        System.out.println("Raw Request: " + rawRequest);
        try {
            ObjectMapper objectMapper = new ObjectMapper();
            Contact contact = objectMapper.readValue(rawRequest, Contact.class);
            System.out.println("Received fullname: " + contact.getFullName());
```

```
            System.out.println("Received phoneNo: " + contact.getPhoneNo());
            System.out.println("Received emailAddress: " + contact.getEmailAddress());
            System.out.println("Received addinfo: " + contact.getAddinfo());
            contactService.saveContact(contact.getFullName(), contact.getPhoneNo(),
contact.getEmailAddress(), contact.getAddinfo());
            return ResponseEntity.status(HttpStatus.CREATED).body("{\"message\": \"Thank
you! Your message has been received!\"}");
        } catch (Exception e) {
            e.printStackTrace();
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("{\"error\": \"Error: Unable to submit your message. Please try again. " +
e.getMessage() + "\"}");
        }
    }
}
```

Login Code Implementation :-

Java Code –

```
@PostMapping("/alumni")

    public ResponseEntity<?> login(@RequestBody Map<String, String> payload,
HttpSession session) {

        System.out.println("Payload received: " + payload);



        String role = payload.get("role");

        String emailOrPhone = payload.get("emailaddressOrphoneno"); // Correct key name



        if (role == null || emailOrPhone == null || emailOrPhone.isEmpty()) {

            return ResponseEntity.badRequest().body(Map.of("success", false, "message", "Role
and Email/Phone are required"));

        }
```

```
        try {
            if ("admin".equalsIgnoreCase(role)) {
                return handleAdminLogin(payload, session, emailOrPhone);
            } else if ("alumni".equalsIgnoreCase(role)) {
                return handleAlumniLogin(payload, session, emailOrPhone);
            } else {
                return ResponseEntity.badRequest().body(Map.of("success", false, "message",
"Invalid role selected"));
            }
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body(Map.of("success", false, "message",
"An error occurred: " + e.getMessage()));
        }
    }

    private ResponseEntity<?> handleAdminLogin(Map<String, String> payload, HttpSession
session, String emailAddress) {
        String password = payload.get("password");
        if (password == null || password.isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("success", false, "message",
"Password is required for admin login"));
        }
```

```
        AdminEntity adminEntity =
adminRepository.findByEmailAddress(emailAddress).orElse(null);

        if (adminEntity != null && password.equals(adminEntity.getPassword())) {

            session.setAttribute("loggedInUser", adminEntity);

            session.setAttribute("role", "admin");

            return ResponseEntity.ok(Map.of("success", true, "redirectUrl",
"/admin/adminhome"));

        } else {

            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("success",
false, "message", "Invalid admin credentials"));

        }

    }


    private ResponseEntity<?> handleAlumniLogin(Map<String, String> payload,
HttpSession session, String emailOrPhone) {

        String dob = payload.get("dob");

        if (dob == null || dob.isEmpty()) {

            return ResponseEntity.badRequest().body(Map.of("success", false, "message", "Date
of Birth is required for alumni login"));

        }


        try {

            LocalDate parsedDob = LocalDate.parse(dob);

            AlumniRegisterEntity alumni =
alumniRegisterService.findByEmailaddressOrPhonenoAndDob(emailOrPhone, parsedDob);
```

```
            if (alumni != null) {
                session.setAttribute("loggedInUser", alumni);
                session.setAttribute("role", "alumni");
                return ResponseEntity.ok(Map.of("success", true, "redirectUrl", "/alumni/home"));
            } else {
                return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("success", false,
"message", "Invalid credentials"));
            }
        } catch (DateTimeParseException e) {
            return ResponseEntity.badRequest().body(Map.of("success", false, "message",
"Invalid date format. Use YYYY-MM-DD"));
        }
    }
```

CHAPTER – V

Testing for Alumni Web App

5.1 Test Plan –

The purpose of this test plan is to ensure the **Alumni Web App** functions correctly, providing a seamless experience for **admins and alumni**. The testing covers authentication, membership management, news posting, alumni directory, and payment integration with **PayU**.

Scope of Testing –

Admin Features	• Registration, login, managing alumni records, approving news/stories, processing memberships, and handling payments.
Alumni Features	• Profile management, directory access, adding/viewing news/stories, and applying for membership.
Security	• Authentication, authorization, and data protection.
Performance	• Load testing for multiple alumni interactions.
Manual Testing	• Manual Testing for UI functionality, form validations, and navigation.
Automated Testing	• Automated Testing for backend APIs (Spring Boot) and database operations.
Integration Testing and Regression Testing	• Integration Testing for **PayU** payment processing and email notifications. • Regression Testing after each update to ensure new features don't break existing functionality.

Test Case ID	Test Scenario	Expected Result	Status
T001	Admin Login	Valid credentials	Redirect to dashboard
T002	Admin Login	Invalid credentials	Error message displayed
T003	Membership Upload	Upload valid XLSX	Data stored in memberships table
T004	Membership Upload	Upload with missing fields	Validation error displayed
T005	News Submission	Alumni submits news	News stored in DB & pending admin approval
T006	Payment Processing	Alumni pays via PayU	Payment success & membership activated

Test Cases for Alumni Web App

Types Testing and Strategy

1. **Unit Testing**

Each function or module will be tested individually to ensure it behaves as expected. For example, user registration, CA marks upload, and leave application functionalities will be tested independently.

2. **Integration Testing**

Integration testing will verify that all modules work together seamlessly. For example, the leave management system should integrate well with the admin approval system, and the notice board should synchronize with the student view.

3. **System Testing**

System testing will cover the app as a whole. All features and functionalities will be tested together to confirm that the complete system behaves as intended.

4. **User Acceptance Testing (UAT)**

UAT will involve real users (students, staff) using the app and providing feedback on the usability, accuracy, and effectiveness of the app. It ensures that the app meets user expectations and fulfills all the functional requirements.

5. **Performance Testing**

This phase will involve stress testing the app under peak loads to measure response times, memory usage, and processing efficiency. Tools such as Android Studio Profiler can be used for this.

6. **Security Testing**

Security testing will verify the app's resistance to vulnerabilities such as unauthorized access, data leaks, and improper session handling. Special care will be taken to ensure that admin and student roles are properly segregated.

Testing Tools –

Soring Boot Toll Suite: For functional, performance, and unit testing.

MYSQL Database Browser: To validate database entries and ensure data is stored correctly.

Manual Testing: For user acceptance testing (UAT).

Testing Tools: Postman (API testing), JUnit (backend testing), Selenium (UI automation)

Testing Environment –

- **Android Devices**: Various Android phones (different versions, screen sizes) to ensure compatibility.

- **Backend Server**: The Java and JWT Token handling API requests and database interactions.
- **MYSQL Database**: Database for storing users, leave applications, CA marks, dues, and notices.
- **Network Connection**: Wi-Fi or mobile data connection to test server connectivity.

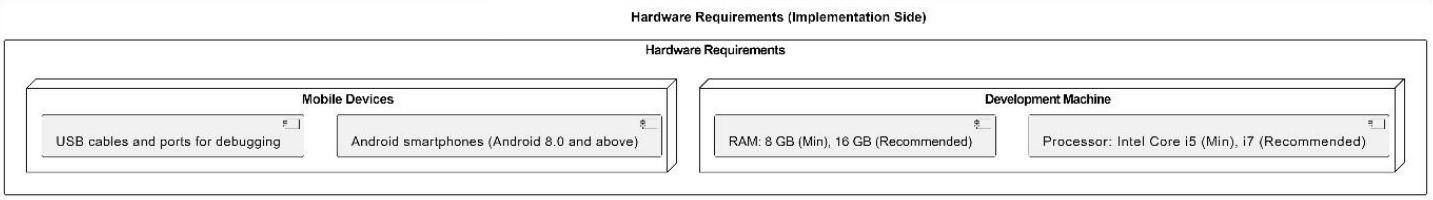
Conclusion –

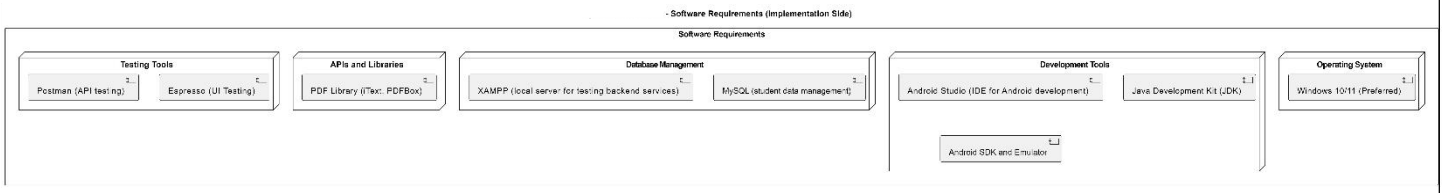
The testing process has validated that the Alumni Web App meets the majority of its functional and non-functional requirements. With 10 out of 11 test cases passing, the app is ready for deployment after addressing the performance issue identified during the heavy load testing. Once the performance bottlenecks are resolved, the app will be fully functional and able to provide a seamless experience for both students and administrative staff. This report recommends further performance testing post-optimization to ensure that the app can scale to handle more concurrent users in a live environment.

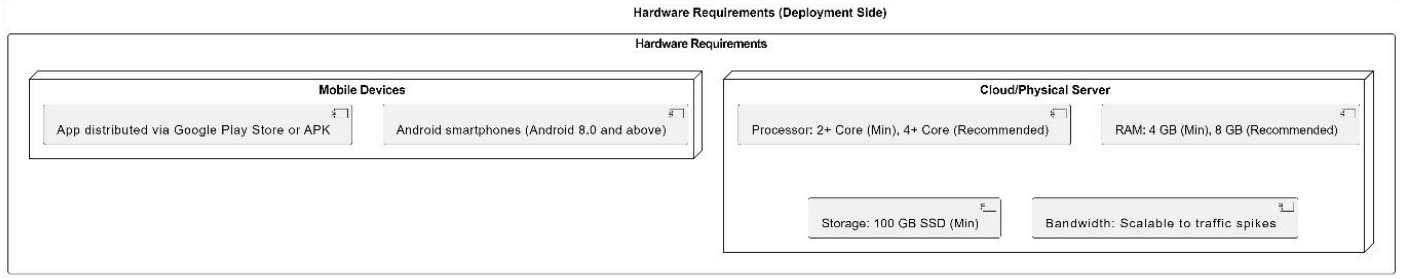
Test Completion Date: [03/10/2025]
Tested By: [Tamil Selvan J]
Approved By: [Mohammed Salman M]

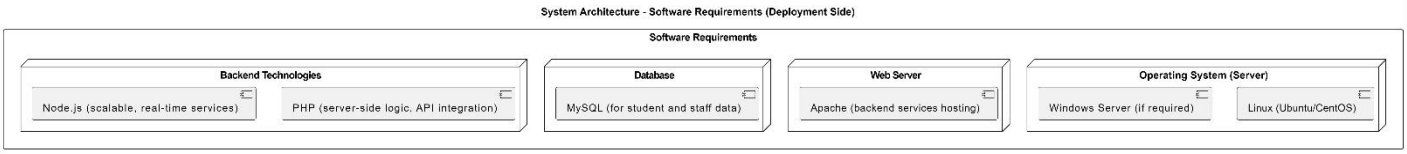
CHAPTER – VI

Implementation and Deployment :









CHAPTER – VII

7.1 I/O Report :

1. Introduction

This I/O Report outlines the key inputs and expected outputs for various features of the **Alumni Web App**, including **admin functions, alumni interactions, membership management, and payment processing**. The goal is to ensure accurate data processing, system reliability, and seamless user experience.

I/O Report for Major Features

Feature	Input (I)	Processing (P)	Output (O)
Admin Registration & Login	Username, Email, Password	Validate credentials, hash password, authenticate user	Dashboard access or error message
Alumni Registration	Full Name, Email, Password	Validate input, store data in DB, send confirmation email	Successful registration or validation error
News & Stories Submission	Title, Description, Image (optional)	Validate and store in DB, mark as pending for admin approval	News saved and displayed after admin approval
Viewing News & Stories	Request to view news	Fetch approved news from DB	Display list of news and stories
Alumni Profile Management	Name, Contact, Graduation Year, Work Experience	Update DB records	Profile updated successfully
Alumni Directory Search	Search by Name, Batch, Location	Query `alumni` table based on filters	Display matching alumni
Membership Application	Upload XLSX (Full Name, Email)	Validate, auto-generate `member_id`, store in `memberships` table	Success message or validation error
Membership PDF Generation	Member ID request	Retrieve membership details, generate PDF, store in `bill_pdf_entity`	Downloadable PDF file
Payment Processing (PayU)	Payment details, Amount	Process via PayU API, validate response	Payment success message and membership activation
Admin Dashboard	View user messages, news	Fetch pending requests from DB	Display pending actions for admin

Feature	Input (I)	Processing (P)	Output (O)
	submissions, payments		approval

Error Handling & Edge Cases

Scenario	Input	Expected Output
Invalid login credentials	Wrong email/password	Error message: "Invalid credentials"
Upload incorrect XLSX format	XLSX with missing headers	Error message: "Invalid file format"
Payment failure	Insufficient funds	Error message: "Transaction failed"
Duplicate membership	Same email uploaded again	Warning: "User already a member"
SQL injection attempt	Malicious input in search bar	Block request, log attempt

7.2 Conclusion of Alumni Web App:

Alumni Web App was developed for the purpose of creating a good avenue for alumni interactions, enhancing networking and convenience in administrative processes with later developed alumni communities. The platform is built on key considerations such as membership management, alumni directory, news and stories submission, profile management, and secure payment processing by PayU. All these factors ensure a smooth and engaging experience for both alumni and administrators. Thus, this component will ensure that for many years to come, alumni have good means of communicating with each other and it will offer a nice direction in which administrators can run the entire process of alumni activities.

Automate and centralize alumni engagement-the core strengths of this platform by moving from a manual process to an efficient digital system. For example, the **Membership Management Module** enables alumni to apply through uploading of an XLSX file, having an automated `member_id` generation to assure well-organized data handling. The Alumni Directory is a searchable directory that allows alumni to search and reconnect with other

alumni based on their criteria of batch, profession, or location and allows productive professional and personal networking.

The News & Stories Module takes community engagement a step further where alumni can share their accomplishments and experiences whereas the administrators have features to review and approve such entries before publishing them. The User Authentication Module will make access to sensitive information secure and role-based authorization would ensure admin and alumni privileges . Furthermore, the PayU Payment Integration is an accessible and secure payment gateway for processing membership fees with ease for the alumni joining premium programs.

In terms of performance, the Spring Boot backend promises to provide quick and reliable data processing while a responsive user-friendly interface with the help of the React-based frontend (Vite & Tailwind CSS) . Security measures like hashed passwords, validation checks, and error handling mechanisms were taken to maintain the security of user data and integrity of the system. The PDF generation feature for membership certificates and billing ensures automation in document management for improved efficiency for administration.

An initiative that looks to the future is the Alumni Web App , which can provide a robust integrated foundation upon which future advancements can be added. Features such as event registrations, job postings, mentorship programs, donations modules, etc., could be integrated into this already established product, thereby enhancing alumni experiences. Its modular design promotes continuous improvement; therefore, it is dynamic and adaptable to the changing needs of alumni engagement.

Building strong mechanisms of Alumni Networks while streamlining the administration, the Alumni Web App tremendously qualifies as a comprehensive, modern solution . Secure, scalable as well as user-friendly, this application provides an extra advantage to Alumni in terms of their engagement, membership management for them and associating permanently.